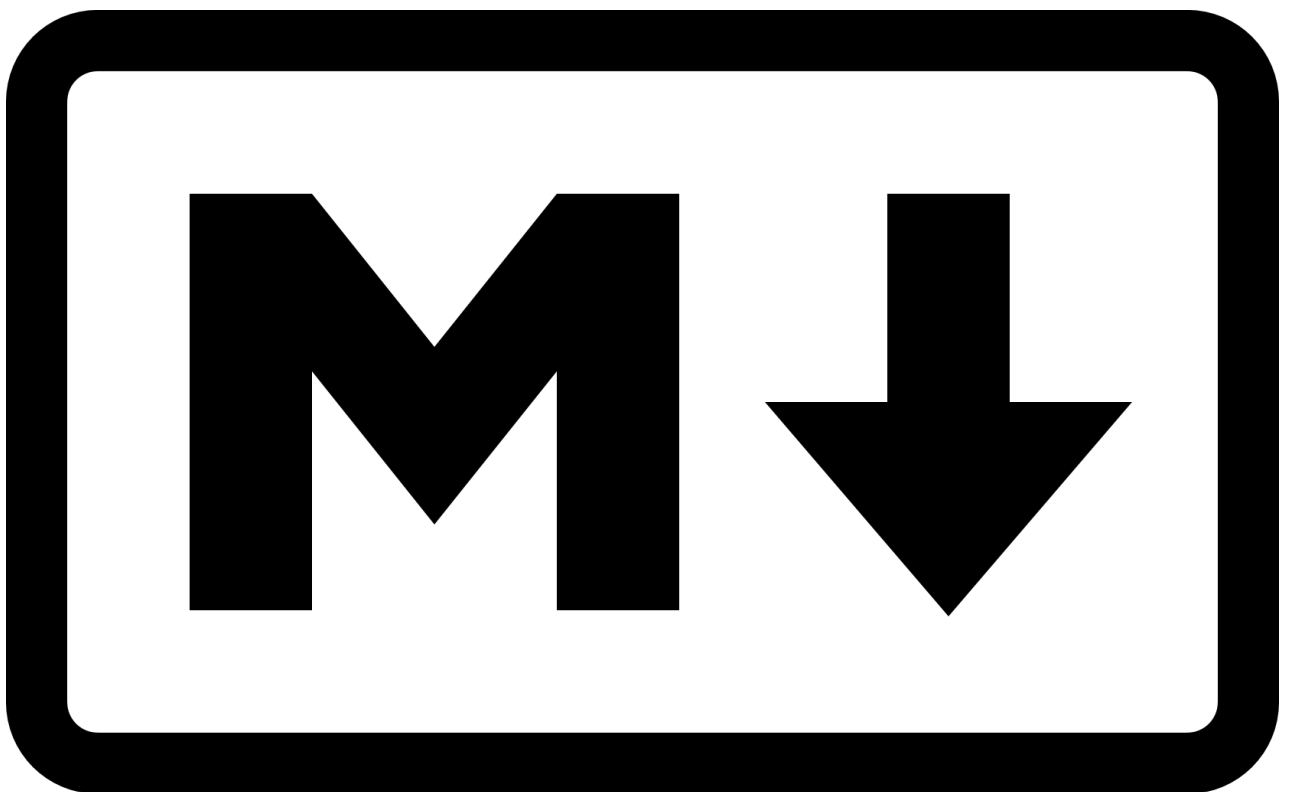


Numpy Introduction

- **NumPy stands for Numerical Python.**
- *NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.*
- It is a library consisting of multidimensional array objects and a collection of routines for processing of array.
- Mathematical and logical operations on arrays.
- Fourier transforms and routines for shape manipulation.
- Operations related to linear algebra. NumPy has in-built functions for linear algebra and random number generation.

Download numpy [numpy \(https://numpy.org/install/\)](https://numpy.org/install/)



Juggling between NumPy and pandas

- The pandas library is actually built on top of NumPy. However, you do need to make a choice between NumPy and pandas when performing tasks. You need the low-level functionality of NumPy to perform some tasks, but pandas makes things so much easier that you want to use it as often as possible.

Knowing when to use NumPy

- It's essential to realize that developers built pandas on top of NumPy. As a result, every task you perform using pandas also goes through NumPy. To obtain the benefits of pandas, you pay a performance penalty that some testers say is 100 times slower than NumPy for a similar task.
- So when speed is essential, NumPy is always the better choice.

Knowing when to use pandas

- You use pandas to make writing code easier and faster. Because pandas does a lot of the work for you.
- The essential consideration, though, is that the pandas library provides rich time-series functionality, data alignment, NA-friendly statistics, groupby, merge, and join methods. Normally, you need to code these features when using NumPy, which means you keep reinventing the wheel.

Why NumPy?

- One of the biggest advantages of using the NumPy package is the ndarray (n-dimensional array) data structure. The NumPy ndarray is much more powerful than the python list, and provides a larger variety of operations and functions than a python array .

Array Creation

```
In [2]: import numpy as np
        np.__version__
```

```
Out[2]: '1.24.3'
```

let's first see how NumPy lets us create an array from a list:

```
In [3]: a=np.array([1,2,3,4,5,6,7,8])
        print(a)
        type(a)
```

```
[1 2 3 4 5 6 7 8]
```

```
Out[3]: numpy.ndarray
```

```
In [4]: # creates an ndarray of 32-bit integer values. The values held in ndarrays will always be of the same type.
        a.dtype
```

```
Out[4]: dtype('int32')
```

```
In [5]: b=np.array([1,2,3.3,4,5.1])
        print(b)
        b.dtype
```

```
[1.  2.  3.3 4.  5.1]
```

```
Out[5]: dtype('float64')
```

NumPy also allows you to explicitly specify the data type of the array when you create it with the dtype argument:

```
In [6]: c=np.array([1,2,3,4,5],dtype=float)
        print(c)
```

```
[1.  2.  3.  4.  5.]
```

NumPy Array Attributes

ndarray.ndim

```
In [7]: a = np.array([[1,2,3],[4,5,6]])
        print(a)
        print(a.ndim)
```

```
[[1 2 3]
 [4 5 6]]
2
```

ndarray.shape

- It returns a tuple of the dimension of the array, i.e. (n,m), where n is number of rows and m is the number of columns

```
In [8]: print(a.shape)
```

```
(2, 3)
```

ndarray.size

- It returns the total number of elements of the array.

```
In [9]: print(a.size)
```

```
6
```

ndarray.itemsize

- It returns the size in bytes of each element of the array.

```
In [10]: print(a.itemsize)
```

```
4
```

Numpy Functions

- NumPy allows the creation of an array of a specified size with placeholder values:

```
In [11]: a=np.zeros((3,3),dtype=int)
print(a)
```

```
[[0 0 0]
 [0 0 0]
 [0 0 0]]
```

```
In [12]: a=np.ones((3,3),dtype=int)
print(a)
```

```
[[1 1 1]
 [1 1 1]
 [1 1 1]]
```

```
In [13]: a=np.full((3,3),22)
print(a)
```

```
[[22 22 22]
 [22 22 22]
 [22 22 22]]
```

Arrays can also be initialized with random values. Using `np.random.random(s)` you can create an array of size `s` filled with random values between 0 and 1.

```
In [14]: np.random.random(5)
```

```
Out[14]: array([0.06835834, 0.81165234, 0.41629354, 0.26628813, 0.32406841])
```

```
In [15]: np.random.random((3,2))
```

```
Out[15]: array([[0.93011321, 0.25523783],
 [0.39836985, 0.19425899],
 [0.73492871, 0.11418386]])
```

`rand(p1,p2,...,pn)` function will create n-dimensional array with random data using uniform distribution, if we do not specify any parameter it will return random float number.

```
In [35]: r1 = np.random.rand()
print(r1)
r2 = np.random.rand(3,2) # no tuple
print(r2)
```

```
0.3140103234137701
[[0.4945461 0.58410826]
 [0.81524455 0.27891828]
 [0.88541581 0.70642517]]
```

`randn(p1,p2,...,pn)` function will create n-dimensional array with random data using standard normal distribution, if we do not specify any parameter it will return random float number.

```
In [36]: r1 = np.random.randn()
print(r1)
r2 = np.random.randn(3,2) # no tuple
print(r2)
```

```
1.644411233197945
[[-0.46623211 0.08476977]
 [-0.77153145 0.93429621]
 [-0.31345116 -0.48977308]]
```

Note: `rand` function will generate random number using uniform distribution, whereas `randn` function will generate random number using standard normal distribution. We are going to learn the difference using visualization technique (as a data scientist, We have to use visualization techniques to convince the audience)

If you want an array with random integer values use `np.random.randint(min, max, size)` . Specify the minimum value for `min` , the maximum value for `max` , and of course the size of the array for `size` just as we did with `np.random.random()` .

```
In [16]: np.random.randint(0,25,(3,3))
```

```
Out[16]: array([[16, 19, 8],
 [14, 21, 23],
 [ 6, 7, 22]])
```

```
In [17]: np.random.randint(-5,5,10)
```

```
Out[17]: array([ 1, -2, -2, -3,  2,  3,  1, -4,  4, -1])
```

Array Manipulation

- The ndarray object allows us to perform arithmetic operations element-wise on two arrays of the same size:

```
In [18]: a=np.array([56,120,80,450])
b=np.array([20,5,6,211])
print(a-b)
print(a+b)
print(a*b) # * is for element wise multiplication

[ 36 115  74 239]
[ 76 125  86 661]
[ 1120  600  480 94950]
```

```
In [19]: a=np.array([[2,3,4],[2,3,4]])
b=np.array([[2,3],[3,4],[4,5]])
print(a@b) # @ is for matrix multiplication

[[29 38]
 [29 38]]
```

Numpy provides various aggregation functions to perform operations on arrays. Aggregation functions in NumPy allow you to perform computations across the entire array or along a specified axis. Here are some commonly used NumPy aggregation functions:

- min/max
- sum
- mean
- median

```
In [20]: a= np.array([[1,5,6],
                    [4,7,0],
                    [2,9,3]])
print(a)

[[1 5 6]
 [4 7 0]
 [2 9 3]]
```

```
In [21]: print(a*2)

[[ 2 10 12]
 [ 8 14  0]
 [ 4 18  6]]
```

```
In [47]: print(a.min(axis=1))
print(a.argmin(axis=1)) #Returns the indices of the minimum values along an axis.
print(a.argmin(axis=0))

[1 4]
[0 0]
[0 0 0]
```

```
In [45]: print(a.max(axis=0))
print(a.argmax(axis=0)) #Returns the indices of the minimum values along an axis.

[4 5 6]
[1 1 1]
```

```
In [38]: print(a.sum())
print(a.sum(axis=1))
print(a.sum(axis=0))

21
[ 6 15]
[5 7 9]
```

```
In [41]: print(a.mean())
print(a.mean(axis=0))
print(a.mean(axis=1))

3.5
[2.5 3.5 4.5]
[2.  5.]
```

basic statistical values for arrays

```
In [25]: np.mean(a)
```

```
Out[25]: 4.111111111111111
```

```
In [26]: np.std(a)
```

```
Out[26]: 2.7666443551086073
```

```
In [27]: np.var(a)
```

```
Out[27]: 7.65432098765432
```

numpy.arange()

- Syntax

numpy.arange(start, stop, step)

The above function is used to make a numpy array with elements in the range between the start and stop value with the difference of step value.

```
In [28]: import numpy as np
a=np.arange(5,25,4)
print(a)
print(type(a))

[ 5  9 13 17 21]
<class 'numpy.ndarray'>
```

numpy.linspace()

- Syntax

numpy.linspace(start, stop, num_of_elements) function will return evenly spaced numbers over a specified interval.

The above function is used to make a numpy array with elements in the range between the start and stop value and num_of_elements as the size of the numpy array. The default dtype of numpy array is float64

```
In [29]: import numpy as np
a=np.linspace(5,25,10)
print(a)
print(type(a))

[ 5.          7.22222222  9.44444444 11.66666667 13.88888889 16.11111111
 18.33333333 20.55555556 22.77777778 25.         ]
<class 'numpy.ndarray'>
```

eye(n) function will create 2-D NumPy array with ones on the diagonal and zeros elsewhere.

```
In [30]: b = np.eye(3)
print(b)

[[1.  0.  0.]
 [0.  1.  0.]
 [0.  0.  1.]]
```

Reshaping Elements

numpy.reshape()

numpy.reshape(dimensions)

The above function is used to change the dimension of a numpy array. The number of arguments in the reshape decides the dimensions of the numpy array.

shape is int or tuple of ints The new shape should be compatible with the original shape. If an integer, then the result will be a 1-D array of that length. One shape dimension can be -1. **In this case, the value is inferred from the length of the array and remaining dimensions.**

```
In [17]: a = np.arange(6)
print(a)

[0 1 2 3 4 5]
```

```
In [19]: a=a.reshape((2,3))
print(a)

[[0 1 2]
 [3 4 5]]
```

```
In [ ]: b=np.arange(12).reshape((3,4))
print(b)

[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

```
In [34]: a = np.array([[1,2,3], [4,5,6]])
print(np.reshape(a, (3,-1)))
print(np.reshape(a, (-1,2)))

[[1 2]
 [3 4]
 [5 6]]
[[1 2]
 [3 4]
 [5 6]]
```

Indexing and Slicing

Integer Indexing

```
In [ ]: a=np.arange(10)
print(a)
print(a[0])
print(a[:4])
```

```
[0 1 2 3 4 5 6 7 8 9]
0
[0 1 2 3]
```

```
In [ ]: a=np.array([(1,2,3,4),(3,4,5,6),(5,6,7,8)])
print("1==",a)
print("2==",a[0][2])
print("3==",a[:1])
print("4==",a[0:2])
print("5==",a[1,3])
print("6==",a[:2,:2])
print("7==",a[0:2,[1,2]])
print("8==",a[:,::-1])
```

```
1== [[1 2 3 4]
 [3 4 5 6]
 [5 6 7 8]]
2== 3
3== [[1 2 3 4]]
4== [[1 2 3 4]
 [3 4 5 6]]
5== 6
6== [[1 2]
 [3 4]]
7== [[2 3]
 [4 5]]
8== [[4 3 2 1]
 [6 5 4 3]
 [8 7 6 5]]
```

Boolean Indexing

```
In [ ]: a=np.arange(10)
print(a[a>5])
```

```
[6 7 8 9]
```

```
In [ ]: a = np.array([np.nan, 1,2,np.nan,3,4,5])
print(a[~np.isnan(a)])
```

```
[1. 2. 3. 4. 5.]
```

```
In [ ]: a = np.array([1, 2+6j, 5, 3.5+5j])
print(a[np.iscomplex(a)])
```

```
[2. +6.j 3.5+5.j]
```

Get Elements Based on a Condition--np.where() is used

np.where(cond[, x, y]) – if cond condition is satisfied, return x , else return y .

```
In [ ]: # np.where( ) returns a 1 at the indices that hold values greater than 25, and 0 where they are 25 or Less.
a=np.array([(1,2,3,4),(3,4,5,6),(5,6,7,8)])
np.where(a>5,1,0)
```

```
Out[39]: array([[0, 0, 0, 0],
 [0, 0, 0, 1],
 [0, 1, 1, 1]])
```

```
In [ ]: # If you wanted to find Low values (e.g. < 15) and replace them with a -1:
np.where(a<5,-1,a)
```

```
Out[40]: array([[ -1,  -1,  -1,  -1],
 [-1,  -1,   5,   6],
 [ 5,   6,   7,   8]])
```

Using flatten() Method:

- We have used the flatten() method in the code below to convert the 2D array 'arr' into a 1D array.

```
In [ ]: arr = np.array([[78, 98, 434, 554], [232, 45, 66, 34], [23, 89, 00, 365]])
print("Array before using flatten: \n", arr)
print("Array after using flatten: \n", arr.flatten())
```

```
Array before using flatten:
[[ 78  98 434 554]
 [232  45  66  34]
 [ 23  89   0 365]]
Array after using flatten:
[ 78  98 434 554 232  45  66  34  23  89   0 365]
```

Vertical & Horizontal Stacking

- Next, if you want to concatenate two arrays and not just add them, you can perform it using two ways – vertical stacking and horizontal stacking.

```
In [ ]: x= np.array([[1,2],[3,4]])
y= np.array([[5,6],[7,8]])
print(np.vstack((x,y)))
print(np.hstack((x,y)))
```

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
[[1 2 5 6]
 [3 4 7 8]]
```

numpy.sort()

```
In [ ]: a = np.array([[3,7],[9,1]])
print(a)
print(np.sort(a))
print(np.sort(a,axis=0))
```

```
[[3 7]
 [9 1]]
[[3 7]
 [1 9]]
[[3 1]
 [9 7]]
```

```
In [ ]: dt = np.dtype([('name', 'S10'),('age', int)])
a = np.array([("raju",21),("anil",25),("ravi", 17), ("amar",27)],dtype=dt)
print(np.sort(a, order = 'name'))
```

```
[(b'amar', 27) (b'anil', 25) (b'raju', 21) (b'ravi', 17)]
```

numpy.append()

```
In [ ]: a = np.array([[1,2,3],[4,5,6]])
print(np.append(a, [7,8,9]) )
```

```
[1 2 3 4 5 6 7 8 9]
```

```
In [ ]: a = np.array([[1,2,3],[4,5,6]])
print(np.append(a, [[7,8,9]],axis=0 ))
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

```
In [ ]: a = np.array([[1,2,3],[4,5,6]])
print(np.append(a, [[7,8],[9,10]],axis=1) )
```

```
[[ 1  2  3  7  8]
 [ 4  5  6  9 10]]
```

numpy.insert()

```
In [ ]: a = np.array([[1,2],[3,4],[5,6]])
# 'Axis parameter not passed. The input array is flattened before insertion.'
print(np.insert(a,3,[11,12]) )
# 'Axis parameter passed. The values array is broadcast to match input array.'
print(np.insert(a,1,[11],axis = 0))
# Broadcast along axis 1
print(np.insert(a,1,11,axis = 1))

[ 1  2  3 11 12  4  5  6]
[[ 1  2]
 [11 11]
 [ 3  4]
 [ 5  6]]
[[ 1 11  2]
 [ 3 11  4]
 [ 5 11  6]]
```

Basic usage of np.delete()

- np.delete() takes three parameters as follows.
 - numpy.delete(arr, obj, axis=None)
 - arr: Input array
 - obj: Row or column number to delete
 - "axis: Axis to delete

```
In [ ]: a = np.arange(12).reshape(3,4)
print(a)
# 'Array flattened before delete operation as axis not used:'
print(np.delete(a,4))
# dele column two
print(np.delete(a,1,axis = 1))

[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[ 0  1  2  3  5  6  7  8  9 10 11]
[[ 0  2  3]
 [ 4  6  7]
 [ 8 10 11]]
```

Numpy ufunc | Universal functions

- These functions include standard trigonometric functions, functions for arithmetic operations, handling complex numbers, statistical functions, etc. Universal functions have various characteristics which are as follows-
 - It performs fast element-wise array operations.
 - It supports various features like array broadcasting, type casting etc.
 - Numpy, universal functions are objects those belongs to numpy.ufunc class.
 - Python functions can also be created as a universal function using frompyfunc library function.
 - Some ufuncs are called automatically when the corresponding arithmetic operator is used on arrays. For example when addition of two array is performed element-wise using '+' operator then np.add() is called internally.

```
In [ ]: # trigonometric function
angles = np.array([0, 30, 45, 60, 90, 180])
radians = np.deg2rad(angles)
print(np.sin(radians))
print(np.sinh(radians))

[0.00000000e+00  5.00000000e-01  7.07106781e-01  8.66025404e-01
 1.00000000e+00  1.22464680e-16]
[ 0.          0.54785347  0.86867096  1.24936705  2.3012989   11.54873936]
```

```
In [ ]: # Statistical functions:
weight = np.array([50.7, 52.5, 50, 58, 55.63, 73.25, 49.5, 45])
print(np.amin(weight), np.amax(weight))
print(np.percentile(weight, 70))
print(np.mean(weight))
print(np.average(weight))

45.0 73.25
55.317
54.3225
54.3225
```

```
In [ ]: # Math functions
x1 = np.arange(9.0).reshape((3, 3))
x2 = np.arange(3.0)
print(np.add(x1,x2))
print(np.subtract(x1, x2))
print(np.power(x1, 3))

[[ 0.  2.  4.]
 [ 3.  5.  7.]
 [ 6.  8. 10.]]
[[0. 0. 0.]
 [3. 3. 3.]
 [6. 6. 6.]]
[[ 0.  1.  8.]
 [27. 64. 125.]
 [216. 343. 512.]]
```

```
In [ ]:
```